

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL AND DEBUGGING

Oleh:

Faisal Tanjung

NIM. 2410817310012

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel and Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Faisal Tanjung
NIM : 2410817310012

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom., M.Kom
NIP. 19971031 202506 1 009

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code.....	7
B. Output Program	26
C. Pembahasan	27
SOAL 2.....	31
A. Pembahasan	31
TAUTAN GIT	32

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Tes Debugging Compose.....	26
Gambar 2. Screenshot Hasil Tes Debugging XML.....	27

DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt Compose Soal 1	7
Tabel 2. Source Code MyApplication.kt Compose Soal 1	7
Tabel 3. Source Code ComicViewModel.kt Compose Soal 1	7
Tabel 4. Source Code ComicDataSource.kt Compose Soal 1	9
Tabel 5. Source Code ComicListItem.kt Compose Soal 1	11
Tabel 6. Source Code CarouselItem.kt Compose Soal 1	14
Tabel 7. Source Code MainActivity.kt XML Soal 1	15
Tabel 8. Source Code ComicViewModelFactory.kt XML soal 1	16
Tabel 9. Source Code ComicViewModel.kt XML soal 1	16
Tabel 10. Source Code HomeFragment.kt XML Soal 1	18
Tabel 11. Source Code DetailFragment.kt XML Soal 1	21
Tabel 12. Source Code activity_main.xml XML Soal 1	22
Tabel 13. Source Code fragment_detail.xml XML Soal 1	22
Tabel 14. Source Code item_comic.xml XML Soal 1	24

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. Install dan gunakan library Timber untuk logging event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
 - e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi.

Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

A. Source Code

Tabel 1. Source Code MainActivity.kt Compose Soal 1

```
1 package com.example.list
2
3 import android.os.Bundle
4 import androidx.activity.compose.setContent
5 import androidx.appcompat.app.AppCompatActivity
6 import
7     com.example.list.presentation.navigation.AppNavigation
8 import com.example.list.ui.theme.ListTheme
9
10 class MainActivity : AppCompatActivity() {
11     override fun onCreate(savedInstanceState: Bundle?) {
12         super.onCreate(savedInstanceState)
13         setContent {
14             ListTheme {
15                 AppNavigation()
16             }
17         }
18     }
19 }
```

Tabel 2. Source Code MyApplication.kt Compose Soal 1

```
1 package com.example.list.base
2
3 import android.app.Application
4 import com.example.list.BuildConfig
5 import timber.log.Timber
6
7 class MyApplication : Application() {
8     override fun onCreate() {
9         super.onCreate()
10         if (BuildConfig.DEBUG) {
11             Timber.plant(Timber.DebugTree())
12         }
13     }
14 }
```

Tabel 3. Source Code ComicViewModel.kt Compose Soal 1

```
1 package com.example.list.presentation.viewmodel
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.ViewModelProvider
5 import com.example.list.data.ComicDataSource
```

```

6 import com.example.list.domain.model.Comic
7 import kotlinx.coroutines.flow.MutableStateFlow
8 import kotlinx.coroutines.flow.StateFlow
9 import kotlinx.coroutines.flow.asStateFlow
10 import timber.log.Timber
11
12 class ComicViewModel(private val category: String) :
    ViewModel() {
13
14     private val _comics =
15     MutableStateFlow<List<Comic>>(emptyList())
16     val comics: StateFlow<List<Comic>> =
17     _comics.asStateFlow()
18
19     private val _navigateToDetail =
20     MutableStateFlow<Int?>(null)
21     val navigateToDetail: StateFlow<Int?> =
22     _navigateToDetail.asStateFlow()
23
24     init {
25         loadComics()
26     }
27
28     private fun loadComics() {
29         if (category == "All") {
30             _comics.value = ComicDataSource.dummyComics
31             Timber.d("Komik yang masuk ke dalam list:
32             ${_comics.value.size}")
33         }
34     }
35
36     fun onComicClicked(comicId: Int) {
37         val selectedComic = _comics.value.find { it.id ==
38         comicId }
39         Timber.d(
40             "Log data dari list yang dipilih ketika
41             berpindah ke halaman Detail: ${selectedComic?.title}"
42         )
43         _navigateToDetail.value = comicId
44     }
45
46     fun onNavigated() {
47         _navigateToDetail.value = null
48     }
49 }

```



```

44 class ComicViewModelFactory(private val category: String)
   : ViewModelProvider.Factory {
45     override fun <T : ViewModel> create(modelClass:
Class<T>): T {
46         if
(modelClass.isAssignableFrom(ComicViewModel::class.java))
{
47             @Suppress("UNCHECKED_CAST")
48             return ComicViewModel(category) as T
49         }
50         throw IllegalArgumentException("Unknown ViewModel
class")
51     }
52 }

```

Tabel 4. Source Code ComicDataSource.kt Compose Soal 1

```

1 package com.example.list.data
2
3 import com.example.list.R
4 import com.example.list.domain.model.Comic
5
6 object ComicDataSource {
7     val dummyComics = listOf(
8         Comic(
9             1,
10            "Kill Blue",
11            "2023",
12            mapOf(
13                "en" to "A new series from the creators
of Kuroko's Basketball Can a legendary assassin repeat
his life as a high school boy?",
14                "id" to "Serial baru dari kreator
Kuroko's Basketball. Bisakah seorang pembunuh legendaris
mengulangi hidupnya sebagai seorang siswa SMA?"
15            ),
16            R.drawable.kill_blue,
17            "https://komiku.org/manga/kill-blue/"
18        ),
19        Comic(
20            2,
21            "My Bias Gets on the Last Train",
22            "2024",
23            mapOf(
24                "en" to "College student Lee Yeewoon
works overtime and takes the last train. Every day Shin
Haein, a woman carrying a guitar, is seen. They find out

```

	that each other's favorite is the indie musician Long Afternoon and become closer.",
25	"id" to "Lee Yeowoon, seorang mahasiswi, bekerja lembur dan naik kereta terakhir. Setiap hari, ia melihat Shin Haein, seorang wanita yang membawa gitar. Mereka mengetahui bahwa musisi indie favorit masing-masing adalah Long Afternoon dan menjadi semakin dekat."), R.drawable.my_bias, "https://comic.naver.com/webtoon/list?titleId=834896"), Comic(3, "The Villain of Destiny", "2022", mapOf("en" to "Gu Zhangge was transported to the Xuanhuan World. The moment he transmigrated, he saw many models surrounding the Lucky Male Protagonist, and he showed utter contempt for him.", "id" to "Gu Zhangge dipindahkan ke Dunia Xuanhuan. Saat ia bertransmigrasi, ia melihat banyak model mengelilingi Protagonis Pria Beruntung, dan ia menunjukkan rasa jijik yang mendalam kepadanya."), R.drawable.villain_destiny, "https://manhwaus.com/manga/me-the-heavenly-destined-villain/chapter-186/"), Comic(4, "Infinite Mage", "2022", mapOf("en" to "Shirone, a child abandoned in a stable and raised as a commoner. A child who awakens to writing with an innate insight, One day while going to town, she encounters a miracle she's been longing for.", "id" to "Shirone, seorang anak yang ditinggalkan di kandang dan dibesarkan sebagai rakyat biasa. Seorang anak yang terbangun dengan bakat menulis yang luar biasa. Suatu hari, saat pergi ke kota, dia menemukan keajaiban yang telah lama dia dambakan."), R.drawable.endless_wizard,
26	
27	
28	
29	
30	
31	
32	
33	
34	
35	
36	
37	
38	
39	
40	
41	
42	
43	
44	
45	
46	
47	
48	
49	

50	"https://mangadex.org/title/4d97b35d-2d9f-4369-964f-3b0ce2c7fbbc/the-infinite-mage?tab=recommendations"
51	),
52	Comic(
53	5,
54	"Kaguya-sama wa Kokurasetai Tensai-tachi no Renai Zunousen",
55	"2015",
56	mapOf(
57	"en" to "Kaguya Shinomiya and Miyuki Shirogane are members of the highly prestigious Shuichi'in Academy's student council, confirming their position as geniuses among geniuses.",
58	"id" to "Kaguya Shinomiya dan Miyuki Shirogane adalah anggota dewan siswa Akademi Shuichi'in yang sangat bergengsi, yang menegaskan posisi mereka sebagai jenius di antara para jenius."
59	),
60	R.drawable.kaguya_sama,
61	"https://mangadex.org/title/37f5cce0-8070-4ada-96e5-fa24b1bd4ff9/kaguya-sama-love-is-war?tab=chapters"
62	)
63	)
64	}

Tabel 5. Source Code ComicListItem.kt Compose Soal 1

1	package com.example.list.presentation.components
2	
3	import android.content.Intent
4	import android.net.Uri
5	import androidx.compose.foundation.Image
6	import androidx.compose.foundation.layout.*
7	import
	androidx.compose.foundation.shape.RoundedCornerShape
8	import androidx.compose.material3.Button
9	import androidx.compose.material3.ButtonDefaults
10	import androidx.compose.material3.Card
11	import androidx.compose.material3.CardDefaults
12	import androidx.compose.material3.MaterialTheme
13	import androidx.compose.material3.Text
14	import androidx.compose.runtime.Composable
15	import androidx.compose.ui.Alignment
16	import androidx.compose.ui.Modifier
17	import androidx.compose.ui.draw.clip
18	import androidx.compose.ui.layout.ContentScale

```

19 import androidx.compose.ui.platform.LocalContext
20 import androidx.compose.ui.res.painterResource
21 import androidx.compose.ui.res.stringResource
22 import androidx.compose.ui.text.font.FontWeight
23 import androidx.compose.ui.text.intl.Locale
24 import androidx.compose.ui.unit.dp
25 import timber.log.Timber
26 import com.example.list.R
27 import com.example.list.domain.model.Comic
28
29 @Composable
30 fun ComicListItem(comic: Comic, onDetailClick: () ->
Unit) {
31     val context = LocalContext.current
32
33     Card(
34         shape = RoundedCornerShape(24.dp),
35         colors = CardDefaults.cardColors(containerColor
= MaterialTheme.colorScheme.surface),
36         modifier = Modifier.fillMaxWidth()
37     ) {
38         Row(modifier = Modifier.padding(12.dp)) {
39             Image(
40                 painter = painterResource(id =
comic.imageResId),
41                 contentDescription = comic.title,
42                 contentScale = ContentScale.Crop,
43                 modifier = Modifier
44                     .size(100.dp, 140.dp)
45                     .clip(RoundedCornerShape(12.dp))
46             )
47
48             Spacer(modifier = Modifier.width(16.dp))
49
50             Column(modifier = Modifier.weight(1f)) {
51                 Row(
52                     modifier = Modifier.fillMaxWidth(),
53                     horizontalArrangement =
Arrangement.SpaceBetween
54                 ) {
55                     Text(
56                         text = comic.title,
57                         fontWeight = FontWeight.Bold,
58                         modifier = Modifier.weight(1f)
59                     )
60                     Text(text = comic.year, style =
MaterialTheme.typography.bodyMedium)

```

```

61         }
62
63         Spacer(modifier = Modifier.height(8.dp))
64
65         Row(modifier = Modifier.fillMaxWidth())
66     {
67         Text(
68             text =
stringResource(R.string.plot),
69             fontWeight = FontWeight.Bold,
70             style =
MaterialTheme.typography.bodySmall
71         )
72         Text(
73             text =
comic.getPlot(Locale.current.language),
74             style =
MaterialTheme.typography.bodySmall,
75             maxLines = 3
76         )
77     }
78
79     Spacer(modifier
Modifier.height(12.dp))
80
81     Row(
82         modifier = Modifier.fillMaxWidth(),
83         horizontalArrangement =
Arrangement.End,
84         verticalAlignment =
Alignment.CenterVertically
85     ) {
86         Button(
87             onClick = {
88                 Timber.d(
89                     "Log      saat      tombol
Explicit Intent ditekan untuk comic: ${comic.title}"
90                 )
91                 val      intent      =
Intent(Intent.ACTION_VIEW, Uri.parse(comic.browserUrl))
92                 context.startActivity(intent)
93             },
94             modifier = Modifier.padding(end
= 8.dp),
95             colors =
ButtonDefaults.buttonColors(

```

95	containerColor	=
	MaterialTheme.colorScheme.primary.copy(alpha = 0.3f),	
96	contentColor	=
	MaterialTheme.colorScheme.onSurface	
97	)	
98	) {	
99	Text(stringResource(R.string.btn_browser))	
100	}	
101	Button(	
102	onClick = {	
103	Timber.d("Log saat tombol	
	Detail ditekan untuk comic: \${comic.title}")	
104	onDetailClick()	
105	},	
106	colors	=
	ButtonDefaults.buttonColors(	
107	containerColor	=
	MaterialTheme.colorScheme.primary,	
108	contentColor	=
	MaterialTheme.colorScheme.onPrimary	
109	)	
110	) {	
111	Text(stringResource(R.string.btn_detail))	
112	}	
113	}	
114	}	
115	}	
116	}	
117	}	

Tabel 6. Source Code CarouselItem.kt Compose Soal 1

1	package com.example.list.presentation.components
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.layout.*
5	import
	androidx.compose.foundation.shape.RoundedCornerShape
6	import androidx.compose.material3.Card
7	import androidx.compose.material3.CardDefaults
8	import
	androidx.compose.material3.ExperimentalMaterial3Api
9	import androidx.compose.material3.MaterialTheme
10	import androidx.compose.runtime.Composable
11	import androidx.compose.ui.Modifier
12	import androidx.compose.ui.layout.ContentScale

13	import androidx.compose.ui.res.painterResource
14	import androidx.compose.ui.unit.dp
15	import com.example.list.domain.model.Comic
16	import timber.log.Timber
17	
18	@OptIn(ExperimentalMaterial3Api::class)
19	@Composable
20	fun CarouselItem(comic: Comic, onItemClick: () -> Unit)
21	{
22	Card(
23	onClick = {
24	Timber.d("Log saat Carousel Item ditekan
25	untuk comic: \${comic.title}")
26	onItemClick()
27	},
28	shape = RoundedCornerShape(24.dp),
29	modifier = Modifier.size(180.dp, 100.dp),
30	colors = CardDefaults.cardColors(containerColor
31	= MaterialTheme.colorScheme.surface)
32	) {
33	Image(
34	painter = painterResource(id =
35	comic.imageResId),
36	contentDescription = comic.title,
37	contentScale = ContentScale.Crop,
	modifier = Modifier.fillMaxSize()
	)
	}
	}

Tabel 7. Source Code MainActivity.kt XML Soal 1

1	package com.example.list
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import com.example.list.databinding.ActivityMainBinding
6	
7	class MainActivity : AppCompatActivity() {
8	
9	private lateinit var binding: ActivityMainBinding
10	
11	override fun onCreate(savedInstanceState: Bundle?) {
12	super.onCreate(savedInstanceState)
13	binding =
14	ActivityMainBinding.inflate(layoutInflater)
	setContentView(binding.root)

15	}
16	}

Tabel 8. Source Code ComicViewModelFactory.kt XML soal 1

1	package com.example.list.ui.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class ComicViewModelFactory(private val appName: String)
7	: ViewModelProvider.Factory {
8	override fun <T : ViewModel> create(modelClass:
9	Class<T>): T {
10	if
11	(modelClass.isAssignableFrom(ComicViewModel::class.java))
12	{
13	@Suppress("UNCHECKED_CAST")
14	return ComicViewModel(appName) as T
15	}
16	throw IllegalArgumentException("Unknown ViewModel
17	class")
18	}
19	}

Tabel 9. Source Code ComicViewModel.kt XML soal 1

1	package com.example.list.ui.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import com.example.list.data.model.Comic
5	import com.example.list.data.source.ComicDataSource
6	import kotlinx.coroutines.flow.MutableStateFlow
7	import kotlinx.coroutines.flow.StateFlow
8	import kotlinx.coroutines.flow.asStateFlow
9	import timber.log.Timber
10	
11	class ComicViewModel(private val appName: String) :
12	ViewModel() {
13	private val _comics =
14	MutableStateFlow<List<Comic>>(emptyList())
15	val comics: StateFlow<List<Comic>> =
16	_comics.asStateFlow()
17	
18	private val _selectedComicId =
19	MutableStateFlow<Int?>(null)


```

17     val      selectedComicId:      StateFlow<Int?>      =
18     _selectedComicId.asStateFlow()
19     private      val      _selectedBrowserUrl      =
20     MutableStateFlow<String?>(null)
21     val      selectedBrowserUrl:      StateFlow<String?>      =
22     _selectedBrowserUrl.asStateFlow()
23
24     init {
25         loadComics()
26     }
27
28     private fun loadComics() {
29         val data = ComicDataSource.dummyComics
30         _comics.value = data
31         Timber.d("Data loaded into list:  ${data.size}
32 items")
33     }
34
35     fun onComicClick(id: Int) {
36         Timber.i("Detail button pressed for comic ID:
37 ${id}")
38
39         val currentList = _comics.value
40         val foundComic = currentList.find { it.id == id
41 }
42
43         if (foundComic != null) {
44             Timber.d("Selected comic data:
45 ${foundComic.title}")
46             _selectedComicId.value = foundComic.id
47         } else {
48             Timber.e("Comic not found in list!")
49         }
50     }
51
52     fun onBrowserClick(url: String) {
53         Timber.i("Explicit Intent (Browser) button
54 pressed for URL:  ${url}")
55         _selectedBrowserUrl.value = url
56     }
57
58     fun onNavigated() {
59         _selectedComicId.value = null
60         _selectedBrowserUrl.value = null
61     }

```

56	fun getAppName() = appName
57	}

Tabel 10. Source Code HomeFragment.kt XML Soal 1

1	package com.example.list.ui.fragment
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import android.view.View
7	import androidx.core.os.bundleOf
8	import androidx.fragment.app.Fragment
9	import androidx.fragment.app.viewModels
10	import androidx.lifecycle.Lifecycle
11	import androidx.lifecycle.lifecycleScope
12	import androidx.lifecycle.repeatOnLifecycle
13	import androidx.navigation.fragment.findNavController
14	import com.example.list.R
15	import com.example.list.databinding.FragmentHomeBinding
16	import com.example.list.ui.adapter.ComicAdapter
17	import com.example.list.ui.adapter.HighlightAdapter
18	import com.example.list.ui.viewmodel.ComicViewModel
19	import
	com.example.list.ui.viewmodel.ComicViewModelFactory
20	import kotlinx.coroutines.launch
21	import timber.log.Timber
22	
23	class HomeFragment : Fragment(R.layout.fragment_home) {
24	private var _binding: FragmentHomeBinding? = null
25	private val binding get() = _binding!!
26	
27	private val viewModel: ComicViewModel by viewModels
28	{
	ComicViewModelFactory(getString(R.string.app_name))
29	}
30	
31	override fun onCreateView(view: View,
	savedInstanceState: Bundle?) {
32	super.onCreateView(view, savedInstanceState)
33	_binding = FragmentHomeBinding.bind(view)
34	
35	setupToolbar()
36	setupRecyclerViews()
37	observeViewModel()
38	}
39	

```

40     private fun setupToolbar() {
41         binding.toolbar.title = viewModel.getAppname()
42         binding.toolbar.setOnMenuItemClickListener {
43             if (it.itemId == R.id.action_language) {
44                 findNavController().navigate(R.id.action_home_to_language)
45                 true
46             } else false
47         }
48     }
49
50     private fun setupRecyclerViews() {
51         val comicAdapter = ComicAdapter(
52             onDetailClick = { comic ->
53                 Timber.i("Detail button clicked for:
54                 ${comic.title}")
55                 viewModel.onComicClick(comic.id)
56             },
57             onBrowserClick = { comic ->
58                 Timber.i("Explicit Intent button clicked
59                 for: ${comic.title}")
60                 viewModel.onBrowserClick(comic.browserUrl)
61             }
62         )
63         binding.rvComics.adapter = comicAdapter
64
65         val highlightAdapter = HighlightAdapter(
66             onItemClick = { comic ->
67                 Timber.i("Highlight item clicked:
68                 ${comic.title}")
69                 viewModel.onComicClick(comic.id)
70             }
71         )
72         binding.rvHighlight.adapter = highlightAdapter
73         if (binding.rvHighlight.onFlingListener == null) {
74             androidx.recyclerview.widget.PagerSnapHelper().attachTo
75             RecyclerView(binding.rvHighlight)
76         }
77     }
78
79     private fun observeViewModel() {
80         viewLifecycleOwner.lifecycleScope.launch {

```

```

77 viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
78     launch {
79         viewModel.comics.collect { comics ->
80             if (comics.isNotEmpty()) {
81                 Timber.d("Received
82                 ${comics.size} comics from ViewModel")
83             }
84             (binding.rvComics.adapter as?
85             ComicAdapter)?.submitList(comics)
86             (binding.rvHighlight.adapter as?
87             HighlightAdapter)?.submitList(comics)
88         }
89     }
90     launch {
91         viewModel.selectedComicId.collect {
92             id ->
93                 id?.let {
94                     Timber.d("Navigating to
95                     detail with ID: ${it}")
96                     val bundle =
97                     bundleOf("comicId" to it)
98                     findNavController().navigate(R.id.action_home_to_detail
99                     , bundle)
100                     viewModel.onNavigated()
101                 }
102             }
103         }
104     }
105     launch {
106         viewModel.selectedBrowserUrl.collect { url ->
107             url?.let {
108                 Timber.d("Opening browser
109                 for URL: ${it}")
110                 val intent =
111                 Intent(Intent.ACTION_VIEW, Uri.parse(it))
112                 startActivity(intent)
113                 viewModel.onNavigated()
114             }
115         }
116     }
117 }

```

111	}
112	
113	override fun onDestroyView() {
114	super.onDestroyView()
115	_binding = null
116	}
117	}

Tabel 11. Source Code DetailFragment.kt XML Soal 1

1	package com.example.list.ui.fragment
2	
3	import android.os.Bundle
4	import android.view.View
5	import androidx.fragment.app.Fragment
6	import androidx.navigation.fragment.findNavController
7	import com.bumptech.glide.Glide
8	import com.example.list.R
9	import com.example.list.data.source.ComicDataSource
10	import com.example.list.databinding.FragmentDetailBinding
11	import java.util.Locale
12	import timber.log.Timber
13	
14	class DetailFragment : Fragment(R.layout.fragment_detail) {
15	private var _binding: FragmentDetailBinding? = null
16	private val binding get() = _binding!!
17	
18	override fun onCreateView(view: View,
	savedInstanceState: Bundle?) {
19	super.onCreateView(view, savedInstanceState)
20	_binding = FragmentDetailBinding.bind(view)
21	
22	val comicId = arguments?.getInt("comicId") ?: -1
23	val comic = ComicDataSource.dummyComics.find {
	it.id == comicId }
24	
25	comic?.let {
26	Timber.d("Displaying detail for comic:
	\${it.title}")
27	binding.toolbar.title = it.title
28	binding.toolbar.setNavigationOnClickListener {
	findNavController().popBackStack() }
29	
30	Glide.with(this).load(it.imageResId).into(binding.ivDetail
	)
31	binding.tvTitle.text = it.title
32	binding.tvYear.text = it.year

33	
34	val isId = Locale.getDefault().language == "in"
	Locale.getDefault().language == "id"
35	binding.tvPlot.text = if (isId) it.plotId else
	it.plotEn
36	} ?: run {
37	Timber.e("Comic with ID \${comicId} not found!")
38	}
39	}
40	
41	override fun onDestroyView() {
42	super.onDestroyView()
43	_binding = null
44	}
45	}

Tabel 12. Source Code activity_main.xml XML Soal 1

1	<androidx.fragment.app.FragmentContainerView
2	
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:id="@+id/nav_host_fragment"
5	
	android:name="androidx.navigation.fragment.NavHostFragment"
	"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	app:defaultNavHost="true"
9	app:navGraph="@navigation/nav_graph" />

Tabel 13. Source Code fragment_detail.xml XML Soal 1

1	<androidx.coordinatorlayout.widget.CoordinatorLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
2	xmlns:app="http://schemas.android.com/apk/res-auto"
3	android:layout_width="match_parent"
4	android:layout_height="match_parent"
5	android:background="#1E1925">
6	
7	<com.google.android.material.appbar.AppBarLayout
8	android:layout_width="match_parent"
9	android:layout_height="wrap_content"
10	android:background="#1E1925">
11	
12	
	<com.google.android.material.appbar.MaterialToolbar
13	android:id="@+id/toolbar"

```

14         android:layout_width="match_parent"
15         android:layout_height="?attr/actionBarSize"
16         app:navigationIcon="@drawable/ic_back"
17         app:titleTextColor="@android:color/white" />
18     </com.google.android.material.appbar.AppBarLayout>
19
20     <androidx.core.widget.NestedScrollView
21         android:layout_width="match_parent"
22         android:layout_height="match_parent"
23
24         app:layout_behavior="@string/appbar_scrolling_view_behavior">
25
26         <LinearLayout
27             android:layout_width="match_parent"
28             android:layout_height="wrap_content"
29             android:orientation="vertical"
30             android:padding="16dp">
31
32             <com.google.android.material.card.MaterialCardView
33                 android:layout_width="match_parent"
34                 android:layout_height="wrap_content"
35                 app:cardCornerRadius="16dp"
36                 app:cardElevation="0dp"
37                 app:strokeWidth="0dp">
38
39                 <ImageView
40                     android:id="@+id/ivDetail"
41                     android:layout_width="match_parent"
42                     android:layout_height="400dp"
43                     android:scaleType="centerCrop" />
44
45                 </com.google.android.material.card.MaterialCardView>
46
47                 <TextView
48                     android:id="@+id/tvTitle"
49                     android:layout_width="match_parent"
50                     android:layout_height="wrap_content"
51                     android:layout_marginTop="24dp"
52                     android:textColor="@android:color/white"
53                     android:textSize="28sp"
54                     android:textStyle="bold" />
55
56                 <TextView
57                     android:id="@+id/tvYear"
58                     android:layout_width="match_parent"

```

57	android:layout_height="wrap_content"
58	android:textColor="#BDBDBD"
59	android:textSize="18sp" />
60	
61	<TextView
62	android:id="@+id/tvPlot"
63	android:layout_width="match_parent"
64	android:layout_height="wrap_content"
65	android:layout_marginTop="16dp"
66	android:lineSpacingExtra="4dp"
67	android:textColor="#EEEEEE"
68	android:textSize="16sp" />
69	</LinearLayout>
70	</androidx.core.widget.NestedScrollView>
71	</androidx.coordinatorlayout.widget.CoordinatorLayout>

Tabel 14. Source Code item_comic.xml XML Soal 1

1	<com.google.android.material.card.MaterialCardView
2	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="wrap_content"
6	android:layout_marginHorizontal="16dp"
7	android:layout_marginVertical="8dp"
8	app:cardBackgroundColor="#2A2431"
9	app:cardCornerRadius="16dp"
10	app:cardElevation="0dp"
11	app:strokeWidth="0dp">
12	<LinearLayout
13	android:layout_width="match_parent"
14	android:layout_height="wrap_content"
15	android:orientation="horizontal"
16	android:padding="12dp">
17	
18	<com.google.android.material.card.MaterialCardView
19	android:layout_width="100dp"
20	android:layout_height="140dp"
21	app:cardCornerRadius="12dp"
22	app:cardElevation="0dp"
23	app:strokeWidth="0dp">
24	
25	<ImageView
26	android:id="@+id/ivComic"
27	android:layout_width="match_parent"
28	android:layout_height="match_parent"
29	android:scaleType="centerCrop" />


```

30 </com.google.android.material.card.MaterialCardView>
31
32     <LinearLayout
33         android:layout_width="0dp"
34         android:layout_height="wrap_content"
35         android:layout_marginStart="16dp"
36         android:layout_weight="1"
37         android:orientation="vertical">
38
39         <RelativeLayout
40             android:layout_width="match_parent"
41             android:layout_height="wrap_content">
42
43             <TextView
44                 android:id="@+id/tvTitle"
45                 android:layout_width="wrap_content"
46                 android:layout_height="wrap_content"
47                 android:layout_alignParentStart="true"
48                 android:layout_toStartOf="@id/tvYear"
49                 android:textColor="@android:color/white"
50                 android:textSize="16sp"
51                 android:textStyle="bold" />
52
53             <TextView
54                 android:id="@+id/tvYear"
55                 android:layout_width="wrap_content"
56                 android:layout_height="wrap_content"
57                 android:layout_alignParentEnd="true"
58                 android:textColor="#BDBDBD" />
59         </RelativeLayout>
60
61         <TextView
62             android:id="@+id/tvPlot"
63             android:layout_width="match_parent"
64             android:layout_height="wrap_content"
65             android:layout_marginTop="8dp"
66             android:ellipsize="end"
67             android:maxLines="3"
68             android:textColor="#EEEEEE" />
69
70         <LinearLayout
71             android:layout_width="match_parent"
72             android:layout_height="wrap_content"
73             android:layout_marginTop="12dp"
74             android:gravity="end"

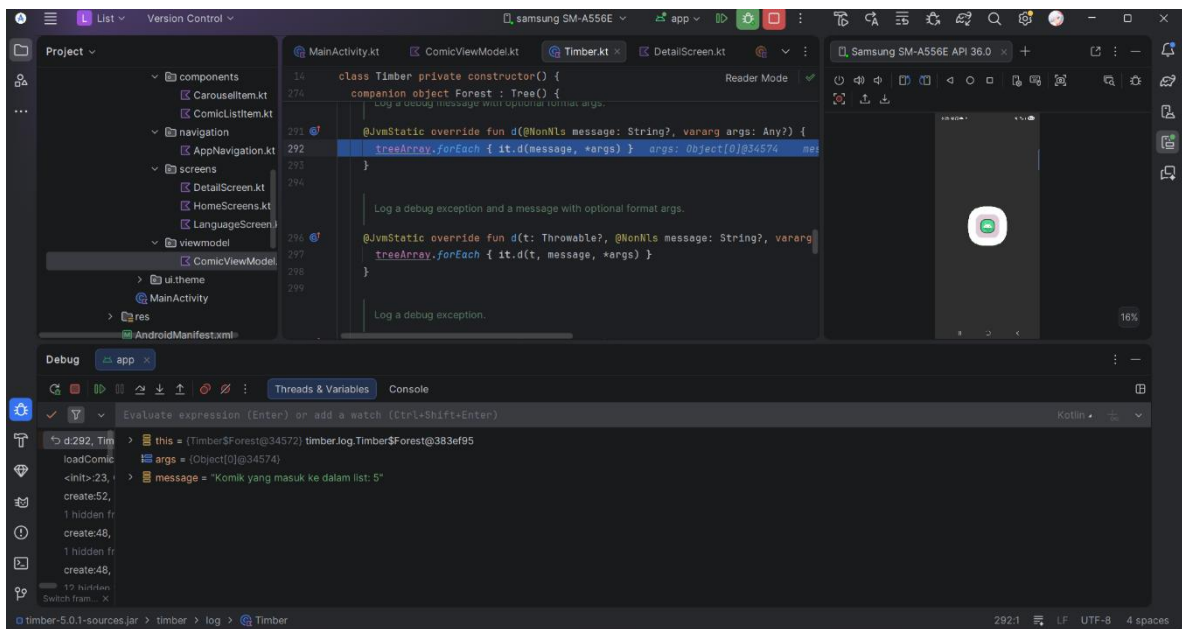
```

```

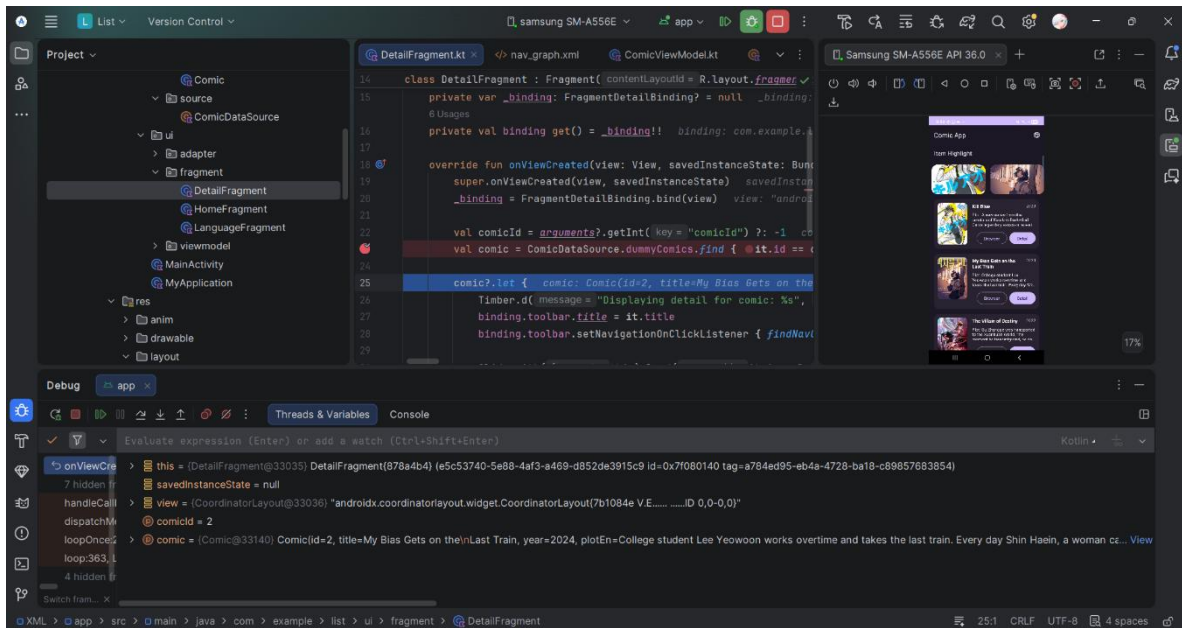
75         android:orientation="horizontal">
76
77         <Button
78             android:id="@+id/btnBrowser"
79
80             style="@style/Widget.Material3.Button.OutlinedButton"
81             android:layout_width="wrap_content"
82             android:layout_height="wrap_content"
83             android:layout_marginEnd="8dp"
84             android:text="@string/btn_browser" />
85
86         <Button
87             android:id="@+id/btnDetail"
88             android:layout_width="wrap_content"
89             android:layout_height="wrap_content"
90             android:text="@string/btn_detail" />
91     </LinearLayout>
92 </LinearLayout>
93 </com.google.android.material.card.MaterialCardView>

```

B. Output Program



Gambar 1. Screenshot Hasil Tes Debungging Compose



Gambar 2. Screenshot Hasil Tes Debugging XML

C. Pembahasan

File ‘MainActivity.kt’ **Compose** merupakan titik masuk utama (*entry point*) aplikasi list item berbasis Jetpack Compose yang di mana kelas ‘MainActivity’ mewarisi ‘AppCompatActivity’ untuk mengelola siklus hidup aplikasi pada sistem Android. Fungsi ‘onCreate()’ bertindak sebagai pintu masuk yang dipanggil saat aplikasi pertama kali dijalankan, yang di dalamnya terdapat fungsi ‘setContent()’ untuk memasang struktur komponen UI langsung ke dalam activity.

File ‘MyApplication.kt’ merupakan main application class yang mewarisi ‘Application’ berfungsi sebagai pusat konfigurasi global yang dieksekusi pertama kali oleh system saat aplikasi dimulai. Class ini mengoverride fungsi ‘onCreate’ untuk menginisialisasi library ‘Timber’ untuk mempermudah proses debugging. Penggunaan ‘BuildConfig.DEBUG’ berfungsi untuk memanggil ‘Timber.plant’ yang akan memastikan bahwa debug tree hanya ditanam selama fase developer.

File ‘ComicViewModel.kt’ berfungsi sebagai state management yang memisahkan logika bisnis dari UI dengan memanfaatkan ‘StateFlow’ untuk mengirimkan pembaruan data. Class ini menggunakan pola pemisahan akses antara ‘MutableStateFlow’ untuk manipulasi data internal dan ‘StateFlow’ publik sebagai read-only state guna memastikan integritas data

saat diobservasi oleh komponen UI. Pada fungsi ‘loadComics’, sistem menginisialisasi data dari ‘ComicDataSource’ berdasarkan parameter kategori, sementara fungsi ‘onComicClicked’ mengelola interaksi user dengan mencatat log aktivitas melalui ‘Timber’ sebelum memperbarui status navigasi.

File ‘ComicDataSource.kt’ bertindak sebagai penyedia data statis yang menyimpan daftar informasi komik melalui variabel ‘dummyComics’ untuk diambil oleh komponen UI aplikasi secara konsisten. Objek ini mengimplementasikan struktur singleton menggunakan declaration ‘object’ untuk memastikan daftar objek ‘Comic’ yang berisi atribut spesifik dapat diakses dari berbagai bagian aplikasi tanpa perlu instansiasi ulang.

File ‘ComicListItem.kt’ berfungsi sebagai komponen antarmuka yang dapat digunakan kembali untuk menampilkan ringkasan informasi setiap judul komik dalam bentuk card. Komponen ini memiliki dua jenis interaksi, yaitu tombol browser yang memicu ‘Intent.ACTION_VIEW’ menggunakan ‘LocalContext.current’ untuk membuka URL eksternal dan tombol detail yang mengeksekusi ‘onDetailClick’ untuk navigasi internal aplikasi. Setiap interaksi tombol akan terjadi pencatatan log melalui ‘Timber’ untuk memastikan setiap interaksi user tercatat dengan baik saat data diambil dari model ‘Comic’ untuk ditampilkan ke monitor.

File ‘CarouselItem.kt’ bertindak sebagai komponen antarmuka visual untuk menampilkan elemen media dalam formal carousel secara ringkas. Komponen ini menggunakan elemen ‘Card’ dengan config ‘RoundedCornerShape’ dan dimensi yang tetap untuk menciptakan container yang konsisten. Anotasi ‘@OptIn(ExperimentalMaterial3Api::class)’ memberikan manfaat seperti interaksi langsung melalui ‘onClick’ yang memicu ‘onItemClick’.

File ‘MainActivity.kt’ **XML** merupakan entry point aplikasi list item berbasis XML yang di mana kelas ‘MainActivity’ mewarisi ‘AppCompatActivity’ untuk mengelola siklus hidup aplikasi pada sistem Android. aplikasi ini dirancang melalui fitur View Binding yang direpresentasikan oleh variabel ‘binding’ yang dimana fungsi ‘ActivityMainBinding.inflate’ bertugas mengubah file layout menjadi objek yang dapat diakses secara langsung tanpa perlu pemanggilan manual yang berulang. ‘setContentView’ memanfaatkan referensi ‘binding.root’ untuk menetapkan hierarki view sebagai akar dari UI.

File ‘DetailFragment.kt’ berfungsi sebagai pengelola tampilan detail melalui pemanfaatan Fragment dan View Binding. Class ini melakukan pengambilan data yang dikirimkan melalui ‘arguments’ untuk kemudian dicocokkan dengan informasi dari ‘ComicDataSource’. Melalui pencatatan log dari ‘Timber’ memastikan setiap interaksi data pada lapisan UI tetap informatif sekaligus mencegah kebocoran memory selama life cycle komponen berlangsung.

File ‘HomeFragment’ bertindak sebagai halaman utama aplikasi yang mengelola daftar item dan konten highlight dengan mengintegrasikan ‘Fragment’, ‘View Binding’, dan ‘View Model’. Class ini menggunakan ‘ComicViewModel’ untuk mengobservasi aliran data secara reaktif melalui ‘lifecycleScope’ dan ‘repeatOnLifecycle’ sehingga pembaruan data pada ‘ComicAdapter’ dan ‘HighlightAdapter’ hanya terjadi saat fragment berada dalam life cycle yang aktif demi efisiensi performa. Fragment ini mengatur interaksi dua arah, yaitu perpindahan layar internal menggunakan ‘findNavController’ berdasarkan perubahan status ‘selectedComicId’ dan eksekusi ‘Intent.ACTION_VIEW’ untuk membuka tautan eksternal melalui peramban secara aman.

File ‘ComicViewModel.kt’ berfungsi sebagai state management dan logika bisnis yang menghubungkan sumber data dengan lapisan UI menggunakan ‘ViewModel’ dan ‘StateFlow’. Class ini mengimplementasi prinsip encapsulation dengan memanfaatkan ‘MutableStateFlow’ untuk mengelola perubahan status internal yang kemudian ditampilkan sebagai read-only state untuk memastikan integritas status aplikasi saat diobservasi oleh fragment. Fungsi ‘onComicClick’ dan ‘onBrowserClick’ bertugas menangani interaksi user dengan memperbarui nilai status diiringi pencatatan log ‘Timber’.

File ‘ComicViewModelFactory.kt’ bertindak sebagai penyedia instance custom yang memungkinkan pembuatan ‘ComicViewModel’ dengan parameter tertentu melalui implementasi antarmuka ‘ViewModelProvide.Factory’ untuk mengatasi constraint constructor standar View Model. Class ini mengoverride fungsi ‘create’ untuk melakukan validasi kesesuaian tipe class yang diminta sehingga argument ‘appName’ dapat diteruskan secara aman ke constructor ViewModel sebelum dikembalikan sebagai objek yang siap digunakan oleh lapisan antarmuka. Dengan prinsip Dependency Injection, factory ini

memastikan bahwa life cycle View Model tetap dikelola sepenuhnya oleh sistem Android meski memerlukan data eksternal saat inisialisasi.

File ‘activity_main.xml’ mendefinisikan ‘FragmentManagerView’ yang berperan sebagai wadah utama (*NavHostFragment*) untuk mengelola alur perpindahan antar layar dalam arsitektur aplikasi berbasis fragmen. Melalui atribut ‘app:navGraph’, komponen ini menghubungkan tata letak aktivitas dengan grafik navigasi ‘@navigation/nav_graph’ yang mendefinisikan rute-rute penting seperti dari ‘HomeFragment.kt’ menuju ‘DetailFragment.kt’. Sementara properti ‘app:defaultNavHost="true"' memastikan interaksi tombol kembali sistem disinkronkan langsung dengan tumpukan fragment.

File ‘fragment_detail.xml’ merupakan tata letak antarmuka tingkat fragmen yang mendefinisikan struktur visual detail komik menggunakan kontainer utama CoordinatorLayout. Struktur ini mengintegrasikan ‘AppBarLayout’ untuk menyediakan toolbar navigasi yang konsisten serta ‘NestedScrollView’ yang memungkinkan pengguna untuk menggulirkan konten informasi secara halus. Di dalam area guliran, penggunaan ‘MaterialCardView’ dengan radius sudut yang lebar memberikan bingkai modern bagi poster komik pada ‘ImageView’, sementara rangkaian ‘TextView’ diatur secara vertikal untuk menyajikan title, tahun, dan alur cerita dengan tipografi yang jelas melalui pengaturan lineSpacingExtra.

File ‘item_comic.xml’ merupakan tata letak komponen yang mendefinisikan tampilan setiap item komik dalam daftar vertikal menggunakan container utama ‘MaterialCardView’ bertema gelap dengan sudut membulat sesuai ketentuan. Struktur ini menggabungkan ‘LinearLayout’ horizontal untuk Menyusun poster komik di sisi kiri dengan kolom informasi di sisi kanan yang mencakup judul dan tahun rilis dalam ‘RelativeLayout’.

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

A. Pembahasan

Application class merupakan sebuah base class yang menjadi entry point utama pada sebuah proses aplikasi. Dia digunakan untuk mengelola global application state atau melakukan inisialisasi yang hanya perlu dilakukan satu kali selama aplikasi tersebut berjalan. Idealnya dia memiliki banyak fungsi, seperti menginisialisasi library seperti firebase atau timber dan melakukan konfigurasi global dengan mengatur tema aplikasi secara dinamis dan mengatur preferensi bahasa. Application class memiliki life cycle yang panjang, dimulai dari saat aplikasi pertama kali dijalankan (created) hingga aplikasi berhenti berjalan oleh system atau pengguna (destroyed).

TAUTAN GIT

Berikut adalah tautan untuk repository GitHub yang telah dibuat:

<https://github.com/Lawdiy/MODUL-MOBILE/tree/61c0e76255630a8fec4b1c1dff4f92727849e237/Modul%204>